

Python-da 2D Numpy

Maqsadlar

Ushbu laboratoriya ishini yakunlaganingizdan so'ng, siz quyidagilarga qodir bo'lasiz:

- `numpy` kutubxonasi bilan bermalol ishlash
- `numpy` yordamida murakkab amallarni bajarish

Mundarija

- 2D Numpy massivini yaratish
- Numpy massivining turli elementlariga murojaat qilish
- Asosiy amallar

2D Numpy massivini yaratish

```
# Kutubxonalarni yuklash
```

```
import numpy as np
```

Uchta **bir xil o'lchamdagi** ichma-ich ro'yxatlarni o'z ichiga olgan `a` ro'yxatini ko'rib chiqing.

```
# Ro'yxat yaratish
```

```
a = [[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

```
a
```

```
[[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

Biz ro'yxatni quyidagicha Numpy massiviga o'tkazishimiz mumkin:

```
# Ro'yxatni Numpy massiviga o'tkazish
```

```
# Har bir element bir xil turga ega
```

```
A = np.array(a)
```

```
A
```

```
array([[11, 12, 13],
       [21, 22, 23],
       [31, 32, 33]])
```

Biz o'qlar yoki o'lchamlar sonini (rank deb ham ataladi) aniqlash uchun `ndim` atributidan foydalanishimiz mumkin.

```
# Numpy massivining o'lchamlarini ko'rsatish
```

```
A.ndim
```

2

`shape` atributi har bir o'lchamning kattaligi yoki elementlar soniga mos keladigan kortejni (tuple) qaytaradi.

```
# Numpy massivining shaklini (o'lchamini) ko'rsatish
```

```
A.shape
```

(3, 3)

Massivdagi elementlarning umumiy soni `size` atributi orqali aniqlanadi.

```
# Numpy massivining hajmini (elementlar sonini) ko'rsatish
```

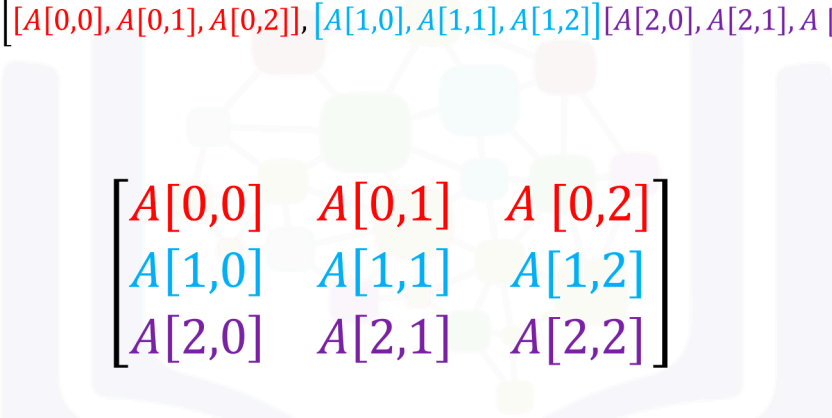
```
A.size
```

9

Numpy massivining turli elementlariga murojaat qilish

Massivning turli elementlariga murojaat qilish uchun to'rtburchak qavslardan foydalanishimiz mumkin. Quyidagi rasmda 3x3 o'lchamli massiv uchun to'rtburchak qavslar, ro'yxat va massiv ko'rinishi o'rtasidagi bog'liqlik ko'rsatilgan:

A: $[[A[0,0], A[0,1], A[0,2]], [A[1,0], A[1,1], A[1,2]], [A[2,0], A[2,1], A[2,2]]]$


$$\begin{bmatrix} A[0,0] & A[0,1] & A[0,2] \\ A[1,0] & A[1,1] & A[1,2] \\ A[2,0] & A[2,1] & A[2,2] \end{bmatrix}$$

Biz 2-qator, 3-ustun elementiga quyidagi rasmda ko'rsatilganidek murojaat qilishimiz mumkin:

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Biz shunchaki to'rtburchak qavslardan va o'zimizga kerakli elementga mos keladigan indekslardan foydalanamiz:

```
# Ikkinchi qator va uchinchi ustundagi elementga murojaat qilish
```

```
A[1, 2]
```

```
np.int64(23)
```

Elementlarni olish uchun biz quyidagi belgidan ham foydalanishimiz mumkin:

```
# Ikkinchi qator va uchinchi ustundagi elementga murojaat qilish
```

```
A[1][2]
```

```
np.int64(23)
```

Quyidagi rasmda ko'rsatilgan elementlarni ko'rib chiqing:

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Biz elementga quyidagicha murojaat qilishimiz mumkin:

```
# Birinchi qator va birinchi ustundagi elementga murojaat qilish
```

```
A[0][0]
```

```
np.int64(11)
```

Numpy massivlarida biz qirqish (slicing) usulidan ham foydalanishimiz mumkin. Quyidagi rasmga e'tibor bering. Biz birinchi qatordagi dastlabki ikkita ustunni olmoqchimiz:

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Buni quyidagi sintaksis yordamida amalga oshirish mumkin:

```
# Birinchi qatorning birinchi va ikkinchi ustunlaridagi elementlarga murojaat qilish
```

```
A[0][0:2]
```

```
array([11, 12])
```

Shuningdek, biz uchinchi ustunning dastlabki ikkita qatorini quyidagicha olishimiz mumkin:

```
# Birinchi va ikkinchi qatorlar hamda uchinchi ustundagi elementlarga murojaat qilish
```

```
A[0:2, 2]
```

```
array([13, 23])
```

Quyidagi rasmga muvofiq:

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Asosiy amallar

Biz massivlarni qo'shishimiz ham mumkin. Bu jarayon matritsalarini qo'shish bilan bir xil. X va Y matritsalarining qo'shilishi quyidagi rasmda ko'rsatilgan:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$X + Y = \begin{bmatrix} 1+2 & 0+1 \\ 0+1 & 1+2 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Numpy massivlari X va Y orqali berilgan:

```
# Numpy massivini yaratish X
```

```
X = np.array([[1, 0], [0, 1]])
X
```

```
array([[1, 0],
       [0, 1]])
```

```
# Numpy massivini yaratish Y
```

```
Y = np.array([[2, 1], [1, 2]])
Y
```

```
array([[2, 1],
       [1, 2]])
```

Numpy massivlarini quyidagicha qo'shishimiz mumkin:

```
# X va Y ni qo'shish
```

```
Z = X + Y  
Z
```

```
array([[3, 1],  
       [1, 3]])
```

Numpy massivini skalyar songa ko'paytirish, matritsani skalyar songa ko'paytirish bilan bir xil. Agar biz `Y` matritsasini 2 skalyar soniga ko'paytirsak, rasmda ko'rsatilganidek, matritsaning har bir elementini shunchaki 2 ga ko'paytiramiz.

$$Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$
$$2Y = \begin{bmatrix} 2 \times 2 & 2 \times 1 \\ 2 \times 1 & 2 \times 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

Numpy'da ham xuddi shu amalni quyidagicha bajarishimiz mumkin:

```
# Numpy massivini yaratish Y
```

```
Y = np.array([[2, 1], [1, 2]])  
Y
```

```
array([[2, 1],  
       [1, 2]])
```

```
# Y ni 2 ga ko'paytirish
```

```
Z = 2 * Y  
Z
```

```
array([[4, 2],  
       [2, 4]])
```

Ikki massivni ko'paytirish elementma-element ko'paytirishga yoki *Adamar ko'paytmasiga* (*Hadamard product*) mos keladi. `X` va `Y` matritsalarini ko'rib chiqing. Adamar ko'paytmasi bir xil pozitsiyadagi elementlarni o'zaro ko'paytirishni anglatadi, ya'ni bir xil rangdagi kataklarda joylashgan elementlar

bir-biriga ko'paytiriladi. Natijada, quyidagi rasmda ko'rsatilganidek, o'lchami **Y** yoki **X** matritsasi bilan bir xil bo'lgan yangi matritsa hosil bo'ladi.

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$
$$X \circ Y = \begin{bmatrix} (1)2 & (0)1 \\ (0)1 & (1)2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

X va **Y** massivlarining elementma-element ko'paytmasini quyidagicha bajarishimiz mumkin:

```
# Numpy massivini yaratish Y
Y = np.array([[2, 1], [1, 2]])
Y
array([[2, 1],
       [1, 2]])
```

```
# Numpy massivini yaratish X
X = np.array([[1, 0], [0, 1]])
X
array([[1, 0],
       [0, 1]])
```

```
# X ni Y ga ko'paytirish
Z = X * Y
Z
array([[2, 0],
       [0, 2]])
```

Numpy massivlari **A** va **B** bilan matritsali ko'paytirishni quyidagicha bajarishimiz mumkin: Dastlab, **A** va **B** matritsalarini aniqlaymiz:

```
# A matritsasini yaratish
A = np.array([[0, 1, 1], [1, 0, 1]])
A
```

```
array([[0, 1, 1],
       [1, 0, 1]])
```

```
# B matritsasini yaratish
```

```
B = np.array([[1, 1], [1, 1], [-1, 1]])
B
```

```
array([[ 1,  1],
       [ 1,  1],
       [-1,  1]])
```

Massivlarni o'zaro ko'paytirish uchun numpy'ning `dot` funksiyasidan foydalanamiz.

```
# Skalyar ko'paytmani (dot product) hisoblash
```

```
Z = np.dot(A, B)
Z
```

```
array([[0, 2],
       [0, 2]])
```

```
# Z ning sinusini hisoblash
```

```
np.sin(Z)
```

```
array([[0.          , 0.90929743],
       [0.          , 0.90929743]])
```

Matritsani transponirlanganini hisoblash uchun biz numpy-ning `T` atributidan foydalanamiz:

```
# C matritsasini yaratish
```

```
C = np.array([[1,1],[2,2],[3,3]])
C
```

```
array([[1, 1],
       [2, 2],
       [3, 3]])
```

```
# C ning transponirlanganini olish
```

```
C.T
```

```
array([[1, 2, 3],
       [1, 2, 3]])
```

2D Numpy massivi bo'yicha viktorina

Quyidagi `a` ro'yxatini ko'rib chiqing va uni Numpy massiviga o'tkazing:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing  
a = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

► Yechimni ko'rish uchun bu yerni bosing

Numpy massivining o'lchamini (elementlar sonini) hisoblang:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing
```

► Yechimni ko'rish uchun bu yerni bosing

Birinchi qator, birinchi va ikkinchi ustundagi elementlarga murojaat qiling:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing
```

► Yechimni ko'rish uchun bu yerni bosing

Numpy massivlari `A` va `B` bilan matritsali ko'paytirishni bajaring:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing  
B = np.array([[0, 1], [1, 0], [1, 1], [-1, 0]])
```

► Yechimni ko'rish uchun bu yerni bosing

Oxirgi mashq!

Tabriklaymiz, siz Python bo'yicha birinchi darsingizni va amaliy mashg'ulotingizni muvaffaqiyatli yakunladingiz.

Numpy bo'yicha amaliy viktorina:

1. Qaysi Python kutubxonasi Pandas uchun asos bo'lib xizmat qiladi va ilmiy hisoblashlar uchun ishlatiladi?

- ☐ OS
- ☐ Numpy
- ☐ Requests
- ☐ datetime

2. Qaysi atribut numpy massividagi elementlar sonini qaytaradi?

- ☐ a.dtype

- ☐ a.size
- ☐ a.shape
- ☐ a.ndim

3. Ushbu massivdagi birinchi elementni qanday qilib 10 ga o'zgartirasiz? `c = np.array([100, 1, 2, 3, 0])`

- ☐ c[1]=10
- ☐ c[0]=10
- ☐ c[2]=10
- ☐ c[4]=10